

工作集模型 (Working Set Model) 能够有效地避免颠簸现象的发生。工作集的定义^[5]是：“一个应用进程在时刻 t 的工作集 W 为它在时间区间 $[t-x, t]$ 使用的内存页集合，也被视为它在未来 (下一个 x 时间内) 会访问的内存页集合。”该模型认为，应当将应用进程的工作集同时保持在物理内存中。因此，早期工作集模型的原则是 all-or-nothing，即一个应用进程的工作集要么全都在物理内存中 (运行时)，要么全都换出，从而减少该应用进程运行时发生换页的次数。现代操作系统很少直接采用 all-or-nothing 作为换页原则，但是工作集的概念依然指导着操作系统的换页策略，即优先将非工作集中的页换出。

那么如何高效地追踪工作集呢？一种常见的实现方法是工作集时钟算法。操作系统设置一个定时器，每经过固定的时间间隔，一个设置好的工作集追踪函数就会被调用。该追踪函数为每个内存页维护两个状态：上次使用时间和访问位，均初始化为 0。每次被调用时，该函数检查每个内存页的状态。如果访问位是 1，则说明在此次时间间隔内该页被访问，于是该函数会把当前系统时间赋值给该内存页的上次使用时间。该方法的前提是 CPU 硬件会在程序访问某个页的时候自动地将对应的访问位 (即页表项中的访问位，例如图 4.6 所示的 AF 位) 设置成 1。如果访问位是 0，则说明在此次时间间隔内该页没有被访问，于是该函数会计算该页的年龄 (当前系统时间 - 该页的上次使用时间)。若该页年龄超过预设时间区间 x ，则它不再属于工作集。检查完一个页的状态之后，工作集追踪函数将其访问位设置为 0。通过工作集时钟算法或者具有相似设计的算法，操作系统能够有效地预测工作集，从而灵活地进行页替换。

5.2.5 利用虚拟内存抽象节约物理内存资源

内存去重

基于写时拷贝机制，操作系统进一步设计了内存去重功能。操作系统可以通过定期地在内存中扫描具有相同内容的物理页面 (全零页是一个典型的例子)，找到映射到这些物理页面的虚拟页面。然后只保留其中一个物理页，并将具有相同内容的其他虚拟页面都用写时拷贝的方式映射到这个物理页，然后释放其他的物理页面以供将来使用。该功能通常是操作系统主动发起使用，对于用户态应用进程完全透明。Linux 操作系统就实现了该功能，称为 KSM (Kernel Same-page Merging)。不过，内存去重功能会对应用进程访存时延造成影响，因为当应用进程写一个被去重的内存页时，既会触发缺页异常，又会导致内存拷贝，从而可能导致性能下降。

当内存去重发生在不同进程之间时，不同进程的虚拟页会映射到同一个物理页，也属于图 4.12 所示的共享内存情形。此外，内存去重也可以发生在同一个进程之中，此时该进程中的多个虚拟页被映射到同一个物理页。

小知识：内存去重的安全问题

内存去重虽然能够有效地节约物理内存资源，但是也可能带来安全问题^[6]。一

个简单的攻击是：攻击者可以在内存中通过穷举的方式不断构造数据，然后等待操作系统去重，再通过访问时延是否变长来窃取数据。比如假设攻击者知道某个进程在内存中保存了一个变量 secret，但不知道这个变量的值。于是攻击者首先构造一个除了变量 secret 外其他都一样的内存页，然后不断改变该变量的值，并记录访问该变量的时延，若时延突然变长，说明这个页发生了去重，即表示猜对了 secret 的值。一种防御这种攻击的可能方法是：操作系统仅在同一用户的应用进程内存之间进行内存去重，从而使得攻击者无法猜测别的用户应用进程中的数据。

内存压缩

为了节约内存资源，现代操作系统也引入了压缩算法来对内存数据进行压缩。其基本原理是：当内存资源不充足的时候，类似“换出”，操作系统选择一些“最近不太会使用”的内存页，压缩其中的数据（但不写入磁盘），从而释放出更多空闲内存。当应用进程访问被压缩的数据时，操作系统将其解压即可，所有操作都在内存中完成。相比于换出内存数据到磁盘上，这样的做法既能够更迅速地腾出空闲内存空间，又能够更快地恢复被压缩的数据。

如图 5.9 所示，Linux 操作系统支持的 zswap 机制是一个使用内存压缩技术的例子。zswap 在内存中为换页过程提供了缓冲区，称为 zswap 区域。操作系统将准备换出的内存数据进行压缩，并且将压缩后的内容写入 zswap 区域（实际上依然是内存）。通过这样的设计，内存数据写到磁盘设备的操作可以延迟完成，从而有可能进行更加高效的磁盘批量 I/O，甚至可能避免磁盘 I/O。即使最后还是会触发磁盘操作，写出 / 读回的数据量由于经过压缩会明显减小。当被换出到 zswap 区域或者磁盘上的数据再次被访问时，该机制会把压缩的数据进行解压和换入。

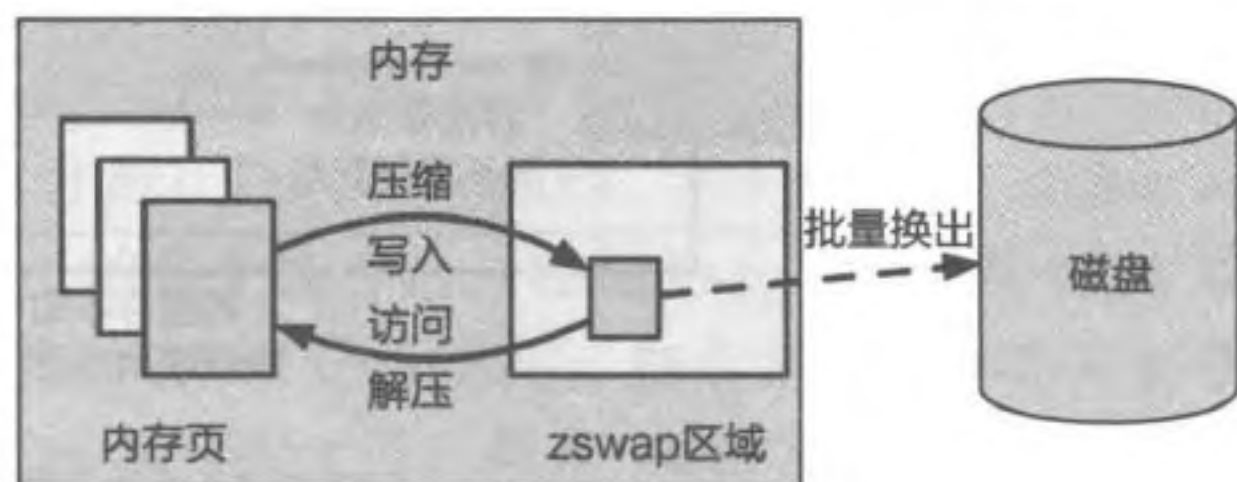


图 5.9 Linux 的 zswap 机制

5.3 性能导向的内存分配扩展机制

本节主要知识点

- 为什么需要 CPU 缓存？CPU 缓存的结构是怎样的？
- 在分配物理内存时操作系统需要考虑 CPU 缓存吗？
- 多核和 NUMA 对于内存分配有什么影响？

5.3.1 物理内存与 CPU 缓存

相比于 CPU 执行的速度，内存访问速度是非常缓慢的：一条算术运算指令可能只需要一个或几个时钟周期即可完成，而一次内存访问则可能需要花费上百个时钟周期。如果每条内存读写指令都需要通过总线访问物理内存，那么 CPU 与物理内存之间的数据搬运可能成为显著的性能瓶颈。

为了降低访存的开销，现代 CPU 内部通常包含 CPU 缓存（CPU Cache）（如图 5.10 所示），用于存放一部分物理内存中的数据。访问 CPU 缓存比访问物理内存快很多，一般最快只需要几个时钟周期。当 CPU 需要向物理内存写入数据时，可以直接写在 CPU 缓存之中；当 CPU 需要从物理内存读取数据时，可以先在 CPU 缓存中查找，如果没找到再去物理内存中获取，并且把取回的数据放入缓存中，以便加速下次读取。由于程序在运行时访问物理内存数据通常具有局部性（包括时间局部性和空间局部性），因此缓存能够有效提升 CPU 访问物理内存数据的性能。

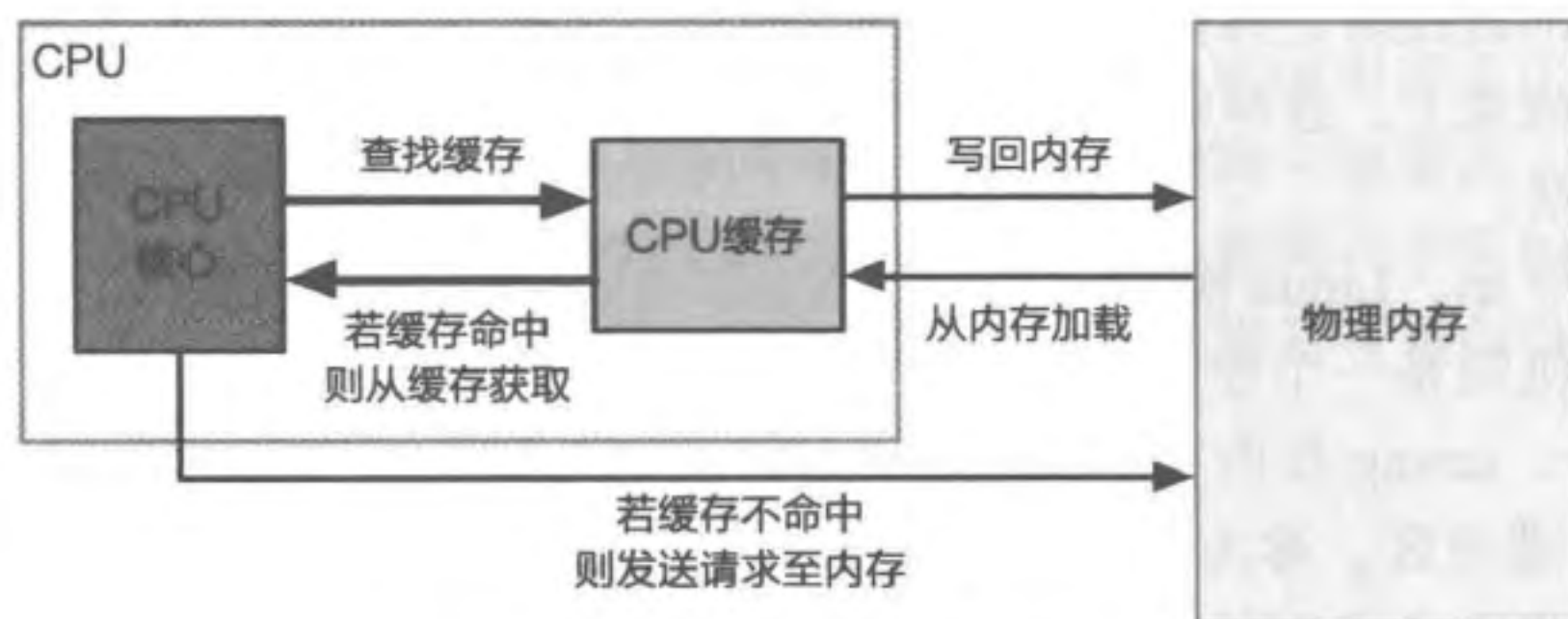


图 5.10 CPU 缓存与内存结构

缓存结构

CPU 缓存中包含若干条缓存行（Cache Line）和每条缓存行相应的状态信息。后者包括：一个有效位（Valid Bit）用于表示该缓存行是否有效，一个标记地址（Tag Address）标识该缓存行对应的物理地址，以及一些其他信息。通常，CPU 以缓存行（常见的是 64 字节，亦作为本章默认缓存行大小）为粒度把物理内存中的数据读取到 CPU 缓存中，也就是说，即使 CPU 实际上只需要物理内存中单个字节的值，也会把物理内存中 64 字节的数据加载到 CPU 缓存中。同样，当数据从 CPU 缓存写回物理内存时，也是以缓存行作为粒度。

典型的 CPU 缓存结构如图 5.11 所示。CPU 是如何在 CPU 缓存中查找数据的呢？CPU 通过 MMU 翻译后的物理地址查找缓存，物理地址在逻辑上分为 Tag、Set（也称为 Index）以及 Offset 三段。组（Set）与路（Way）是 CPU 缓存的经典概念。物理地址的 Set 段能表示的最大数目称为组。例如，如果 Set 段的位数是 8，那么对应的 CPU 缓存的组数就是 256（ 2^8 ）。每组中支持的最大缓存行数（最多的 Tag 数）称为路。例如，图 5.11 中，在 Set 相同的情况下，缓存最多支持 4 个不同的 Tag，也就是 4 路，该 CPU

缓存被称为 4 路组相联（4-Way Set Associative）。

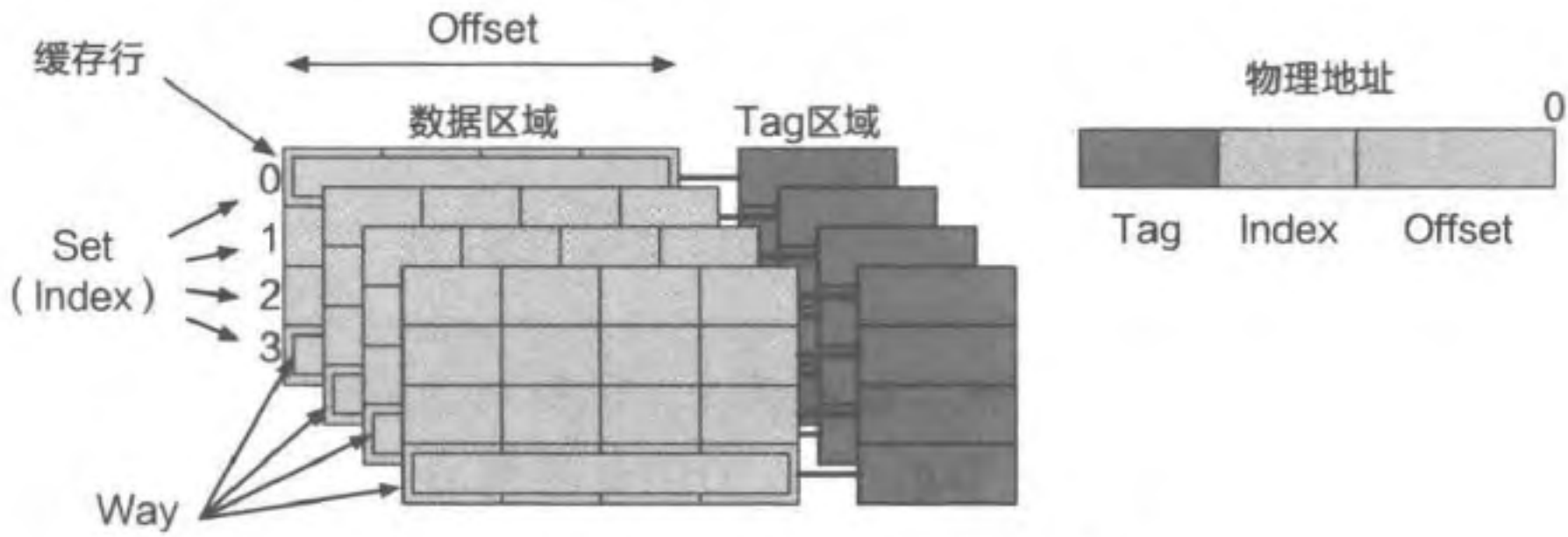


图 5.11 典型的 CPU 缓存结构示意图 [7]

缓存寻址

下面以 Cortex-A57 CPU（AArch64 架构）的 L1 数据缓存为例，介绍 CPU 缓存查找的一般过程。该 CPU 缓存相关的参数 [8] 如下：

- 物理地址长度为 44 位。
- 缓存大小为 32KB，缓存行大小为 64 字节。
- 256 组，2 路组相联缓存。

以图 5.12 为例，假设要读取以物理地址 0x2FBBC030 开始的 4 个字节的物理内存数据。该物理地址按照 Tag、Set 和 Offset 划分是：Tag 为 0xBEEF，Set 为 0x0，Offset 为 0x30（十进制为 48）。根据 Set 定位到 Set=0 的两个缓存行，对比 Tag 并且检查 Valid 是否为 1（表示该缓存行有效），即可进一步根据 Offset 进行访问，在本例中取出的 4 字节字为 23。如果在寻址过程中，虽然 Set 和 Tag 都匹配上了，但是 Valid 为 0，那么该缓存行是无效的，则需从物理内存搬运相应数据到 CPU 缓存中。

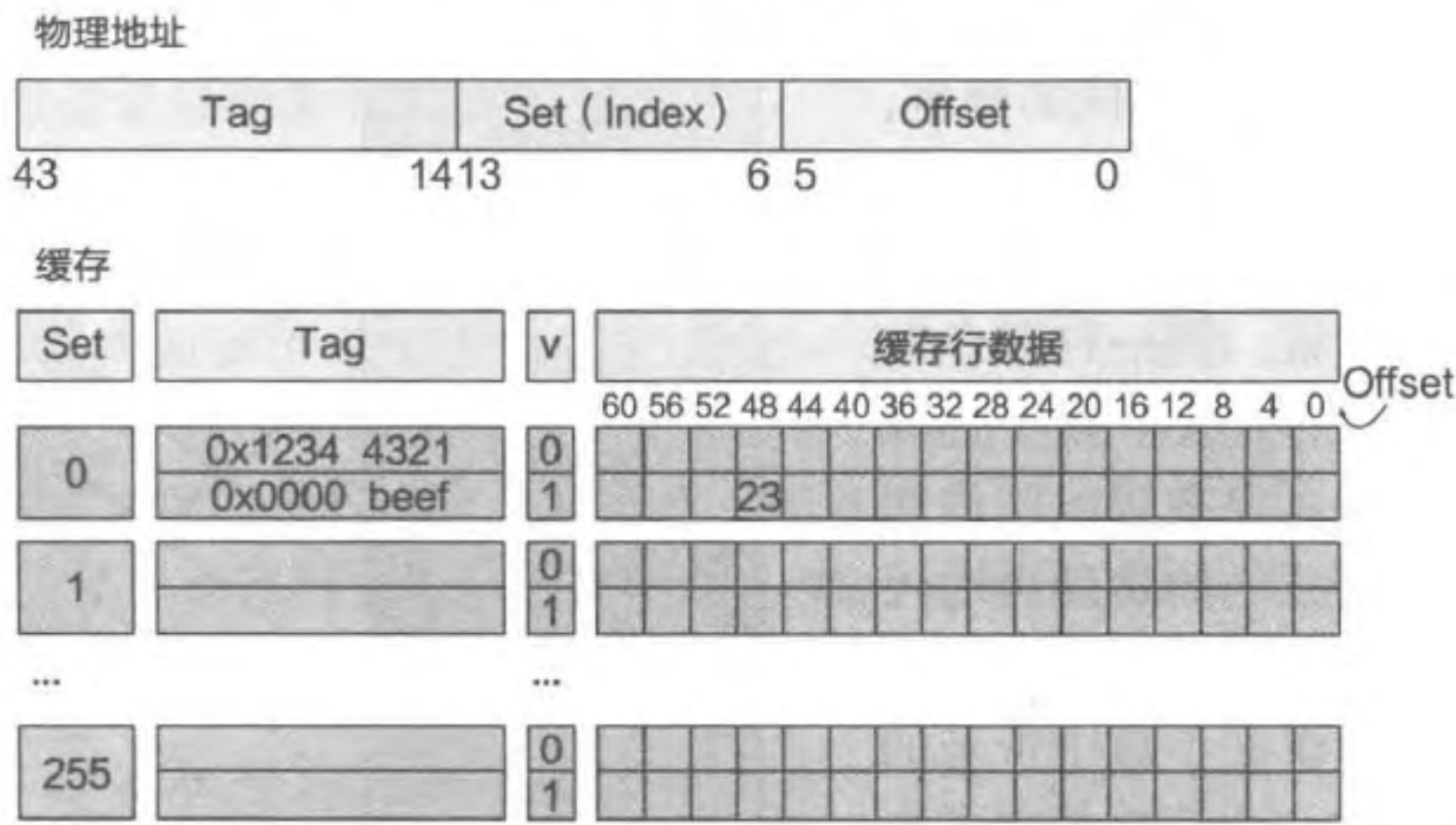


图 5.12 Cortex-A57 CPU 中的 CPU 缓存（L1 数据缓存）结构示意图

CPU 缓存行何时写回物理内存？通常有两种情况：第一，存在专门的硬件指令负责

写回某缓存行，当 CPU 执行这类指令后，相应的缓存行会写回物理内存中；第二，若某组中的缓存行状态都为有效状态且被修改过，而 CPU 需要在该组中加载新的缓存行进行读写，此时 CPU 会首先选择该组中已有的某缓存行写回物理内存（空出一个缓存行）。

练习

5.6 假设 CPU 缓存是 2 路组关联、组数为 8、缓存行大小为 8 字节、物理地址寻址，请问 48 位物理地址中 Tag、Set、Offset 分别占用多少位？请问物理地址 0x3BFE 对应的 Tag、Set、Offset 分别是什么？

5.3.2 物理内存分配与 CPU 缓存

由于访问 CPU 缓存比访问物理内存要快得多，若越多的物理内存数据能被存放在 CPU 缓存中，则程序的性能会得到提升。然而，相比于物理内存，CPU 缓存要小得多，比如一台服务器的物理内存有几百 GB，而 CPU 缓存只有几十 MB。物理内存中的数据会根据物理地址以缓存行为粒度放入 CPU 缓存中，当缓存放不下（某组中的缓存行都被占用）的时候，CPU 则会选择替换某个已有缓存行。操作系统可以根据 CPU 缓存的特点，优化为应用分配物理页的策略，从而更好地利用缓存以提升应用性能。

具体来说，操作系统在给应用进程分配物理页的时候，如果分配的物理页能够尽可能均匀地占用 CPU 缓存中的组，则可以使得更多的应用数据存放到缓存中，从而充分利用缓存大小来提升应用访存性能。为此，操作系统开发者提出了一种称为缓存着色（Cache Coloring / Page Coloring）的内存染色机制。该机制的基本思想很简单：把能够被存放到缓存中不同位置（不造成缓存冲突）的物理页标记上不同的颜色，在为连续虚拟内存页分配物理页面的时候，优先选择不同颜色的物理页面进行分配。换句话说，同一种颜色的物理页面会发生缓存冲突。由于连续的虚拟内存页通常可能在短时间内相继访问，分配不同颜色的物理页面可以让被访问的数据都处于缓存中，不引起冲突从而避免缓存不命中带来的开销。图 5.13 通过一个简单的例子来介绍该机制的实现。假设缓存总大小可以放下 4 个连续的物理页面，则该

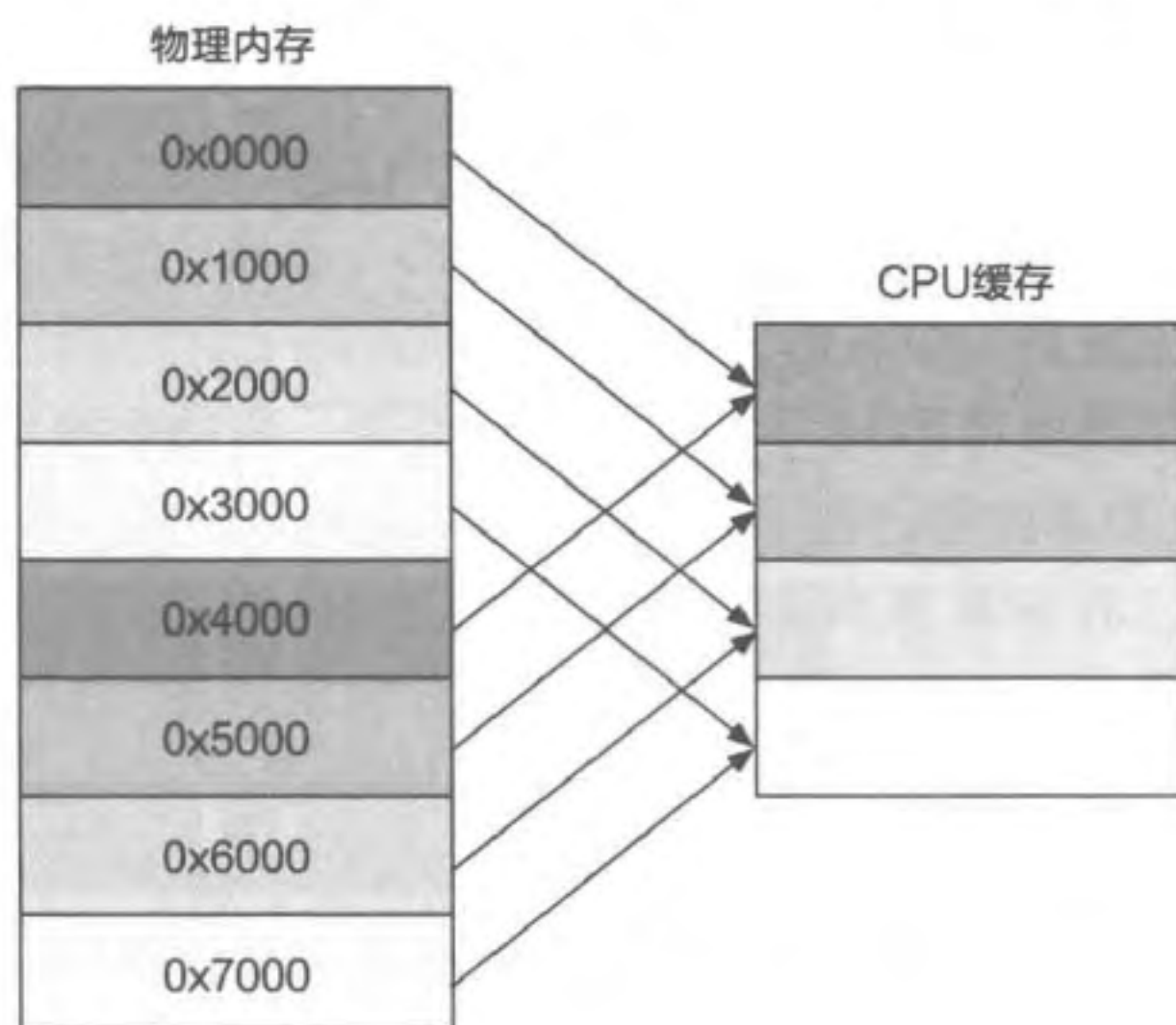


图 5.13 染色机制示意图

机制会把第1个物理页面到第4个物理页面分别染上4种不同的颜色，再把第5个到第8个物理页面的颜色染成与前面4个页面一致的颜色，依次类推。这种染色机制会导致物理页面的分配变得复杂，但如果能够较明确地得知应用对内存的访问模式，则可以有效提升访问内存的性能，FreeBSD、Solaris 等操作系统就运用了该机制^[9]。

5.3.3 多核与内存分配

多核 CPU 包含多个核心，不同的应用可以在不同核心上同时运行，如图 5.14 所示。由于应用进程可能在不同 CPU 核心上同时发起需要在操作系统中分配内存的系统调用，操作系统可能会在不同 CPU 核心上同时调用 SLAB 内存分配接口。为了防止重复分配同一块内存，一种可行的方式是只允许一个 CPU 核心调用 SLAB 内存分配接口，在分配完成后才允许下一个 CPU 核心发起调用请求（该方式可以用第 9 章介绍的锁进行实现，直观上来说，每个 CPU 核心先获取一把锁才能进行内存分配，分配完成后释放锁，而锁的持有者至多只有一个 CPU 核心）。不过，该方式会导致性能下降的问题：当多个 CPU 核心同时需要调用内存分配接口时，实际上每次只有一个 CPU 核心在执行分配操作，而其余 CPU 核心都需要等待它分配结束。如此一来，多核 CPU 的优势就没有体现，因为真正在执行的只有一个 CPU 核心，其余 CPU 核心都在等待，看起来就像是只有单个核心的 CPU。

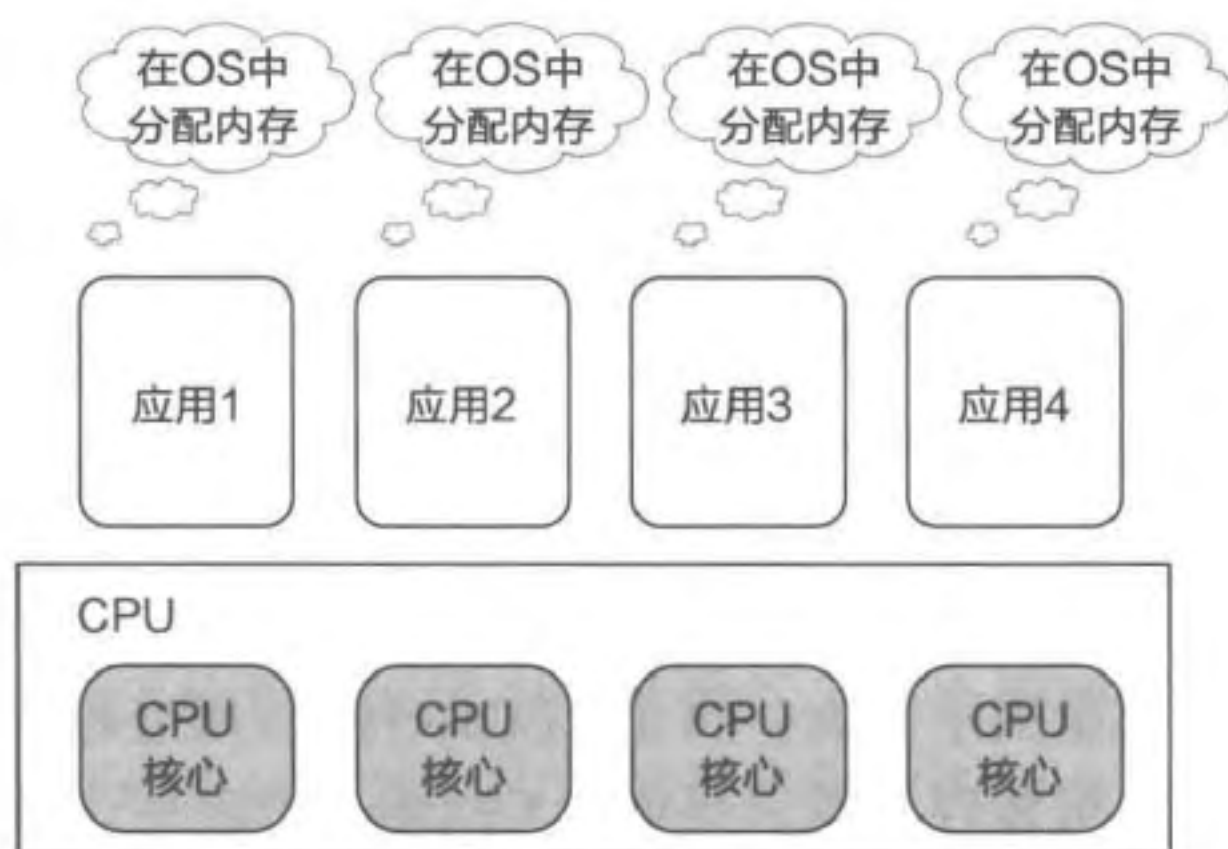


图 5.14 在多核 CPU 上同时运行的应用进程都引起操作系统分配内存的操作

为了满足操作系统在多个 CPU 核心上同时进行内存分配的需求，操作系统开发人员提出了一种为每个 CPU 核心建立内存分配器的设计。比如，在图 5.14 所示的 4 核心 CPU 上，操作系统为每个 CPU 核心都初始化一个 SLAB 分配器（一共 4 个 SLAB 分配器），并根据 CPU 核心调用相应的 SLAB 分配器。通过这样的设计，操作系统就能够在不同的 CPU 核心上同时进行内存分配操作，而不是同一时间只有一个 CPU 核心能够进行分配。类似地，操作系统也可以为每个 CPU 核心初始化一个伙伴系统，从而支持在不同 CPU 核心上并发地分配物理页。

5.3.4 CPU 缓存的硬件划分

在多核 CPU 中，CPU 缓存通常分成多级，每个 CPU 核心拥有私有的 CPU 缓存（例如，L1、L2 缓存），而所有 CPU 核心会共享最末级 CPU 缓存（Last Level Cache, LLC）。

(例如，L3 缓存)。一般来说，CPU 的最末级缓存会被多个 CPU 核心共享。由于每个 CPU 核心可以同时运行不同的应用进程，这些应用进程将会竞争最末级缓存的资源，从而可能由于互相影响导致应用进程产生性能抖动，甚至造成系统整体性能的下降。尤其在云计算多租户场景下，类似的问题更为严重。为此，近年来硬件厂商也开始通过相关硬件特性使得操作系统能够更加灵活地分配 CPU 缓存资源。

Intel CAT

Intel 缓存分配技术 (Cache Allocation Technology, CAT)^[10] 的出现为操作系统有效解决上述问题提供了一种方法。该技术允许操作系统设置应用进程所能使用的最末级缓存的大小和区域，从而实现最末级缓存资源在不同应用进程间的隔离。具体来说，CAT 提供若干服务类 (Class of Service, CLOS)，并允许操作系统把应用进程划分到某个 CLOS 中。每个 CLOS 有一个容量位掩码 (Capacity Bitmask, CBM)，标记着该 CLOS 能够使用的最末级缓存资源：CBM 中设置为 1 的位对应的缓存资源，即为该 CLOS 能使用的最末级缓存资源。因此，CAT 支持按比例限制每个 CLOS 可以使用的最末级缓存大小和区域，不同的 CLOS 对应的缓存资源既可以是完全隔离的，也可以是部分重合的。

以表 5.5 为例，假设硬件共支持 4 个 CLOS，每个 CLOS 有一个总长为 8 位的 CBM。在容量大小方面，属于 CLOS-3 的应用进程的 CBM 所有位均为 1，所以能够使用全部的最末级缓存资源，而属于 CLOS-2 的应用进程由于 CBM 仅有高 4 位被置为 1，故只能占用 50% 的最末级缓存大小，剩余的 CLOS-0 与 CLOS-1 仅被分配到 25% 的最末级缓存。在区域隔离方面，由于 CBM 间不存在重叠的位，CLOS-0 与 CLOS-1 所使用的最末级缓存是完全隔离的，因此 CLOS-0 与 CLOS-1 中的应用进程不会彼此竞争。CLOS-3 有 25% 的独占缓存区域，剩下的区域则可能被不同的 CLOS 竞争。如果把 CLOS-3 中的 Bit-3 和 Bit-2 设为 0，那么 CLOS-1 将会独占 25% 的最末级缓存。

表 5.5 利用 CAT 创建 4 个服务类 CLOS，每个 Bit 对应缓存资源的 12.5%

	Bit-7	Bit-6	Bit-5	Bit-4	Bit-3	Bit-2	Bit-1	Bit-0
CLOS-0			1	1				
CLOS-1					1	1		
CLOS-2	1	1	1	1				
CLOS-3	1	1	1	1	1	1	1	1

操作系统可以通过设置每个 CLOS 的 CBM 并且把不同应用进程划分到不同的 CLOS 中，从而将最末级 CPU 缓存资源分配给不同的应用进程使用。举个例子来说，假设有两个应用进程 A 和 B，操作系统希望它们在同时运行的过程中不干扰对方使用的 CPU 缓存资源，那么可以将它们运行在不同的 CPU 核心上，并且为 A 设置表 5.5 中 CLOS-0，为 B 设置 CLOS-1。

ARMv8-A MPAM

AArch64 架构的 MPAM (Memory System Resource Partitioning and Monitoring) 技术^[11]也可以被操作系统用来实现类似的功能。MPAM 支持配置多个分区 ID (Partition ID，

PARTID)，并可限制每个 PARTID 能够使用的缓存资源。操作系统可以把应用进程划分到某个 PARTID，从而限制该应用进程能够使用的缓存资源。

MPAM 支持两种缓存划分方案，分别是缓存局部划分（Cache-portion Par-titioning）以及缓存最大容量划分（Cache Maximum-capacity Partitioning）。在局部划分中，MPAM 同样使用位图来按比例划分属于不同 PARTID 的可用缓存资源；而在最大容量划分中，MPAM 通过配置 MPAMCFG_CMAX 寄存器的值来设定一个 PARTID 能够使用的最大缓存资源比例。上述两种划分方案可以结合使用，假设共有三个应用进程（对应三个 PARTID），操作系统可以通过局部划分使得优先级最高的进程所在的 PARTID 独占 50% 的缓存资源，而剩余两个 PARTID 共享剩余的 50%。为了限制这两个 PARTID 抢占缓存资源，可以再利用最大容量划分使得它们均不得占用超过共享缓存 75% 的容量（即总缓存资源的 $50\% \times 75\% = 37.5\%$ ）。

5.3.5 非一致内存访问（NUMA 架构）

随着单处理器中核心数量的增多以及多处理器系统的出现，单一的内存控制器（Memory Controller）逐渐成为性能瓶颈^[12]。因此多核及多处理器系统将多个内存控制器分布在不同的核心或处理器上。这种设计在多处理器系统中非常常见，每个处理器往往被分配一个单独的内存控制器。每个核心可以通过本地的内存控制器快速访问本地内存，也可以通过远端的内存控制器访问远端内存。但访问远端内存的时延将远高于访问本地内存的时延。这种由于本地和远端内存而导致访存时延不同的架构被称为非一致内存访问（Non-Uniform Memory Access，NUMA）。

图 5.15 是一个 NUMA 系统的示例，该系统包含两个 NUMA 节点。在这个示例系统中，一个处理器上的多个核心（如核心 0 与核心 1）访问内存的时延特性一致，因此划分到同一个 NUMA 节点（NUMA 节点 0）上。一个节点中的任意核心能够快速访问本节点的本地内存。一旦其需要访问远端其他节点的内存，则需要通过互联总线（Interconnect）与远端节点通信，其时延远远高于访问本地内存。NUMA 架构有多种组成方式，一个 NUMA 节点可以是一个物理处理器，也可以是处理器中的一部分核心。

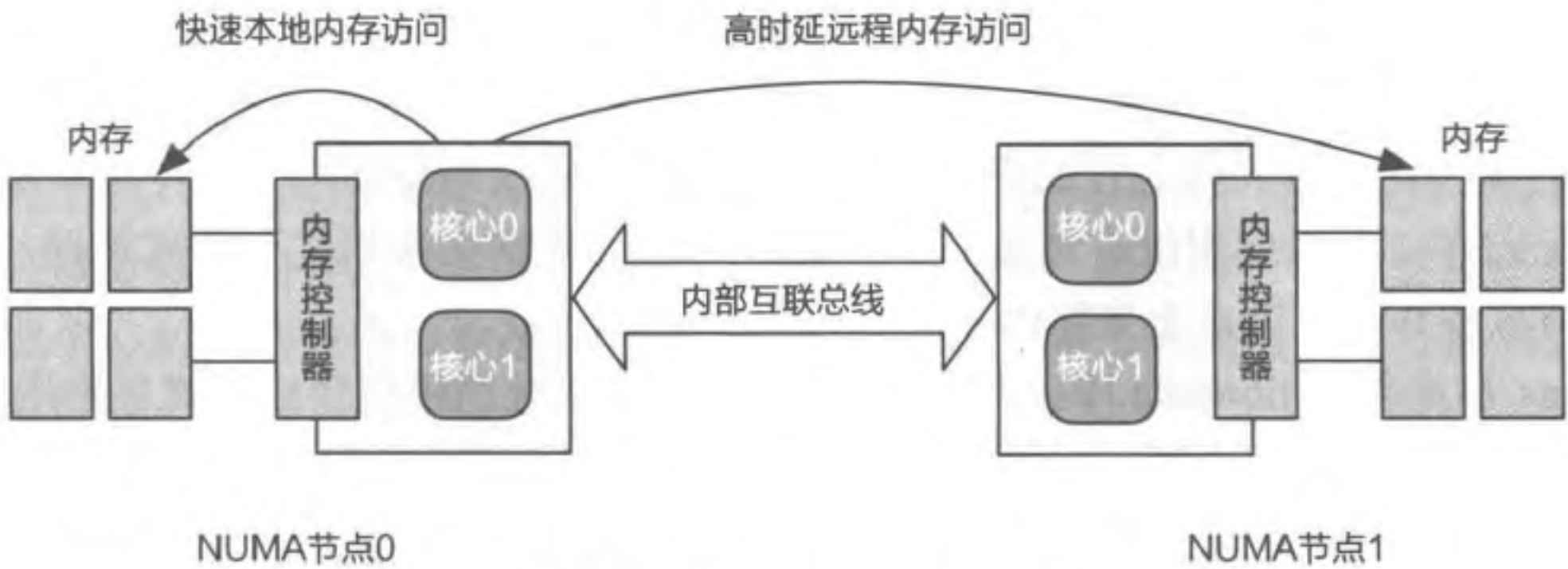


图 5.15 非一致内存访问示例

现代单台服务器为了应对更高的计算需求，可以插入多个处理器，而每个处理器中同时拥有多个核心，因此其 NUMA 架构也更为复杂。图 5.16 展示了 Intel 架构与 AMD 架构的服务器拓扑结构图^[13]。这两个服务器都拥有 8 个处理器插槽，分别对应着 8 个 NUMA 节点。由于节点数量众多，有些节点之间没有直接相连。有的远程内存访问请求甚至需要通过两跳才能到达目标节点（如 Intel Xeon 与 AMD Opteron 服务器中的节点 0 与节点 7，均需要两跳），因此访问时延差异也更加复杂。

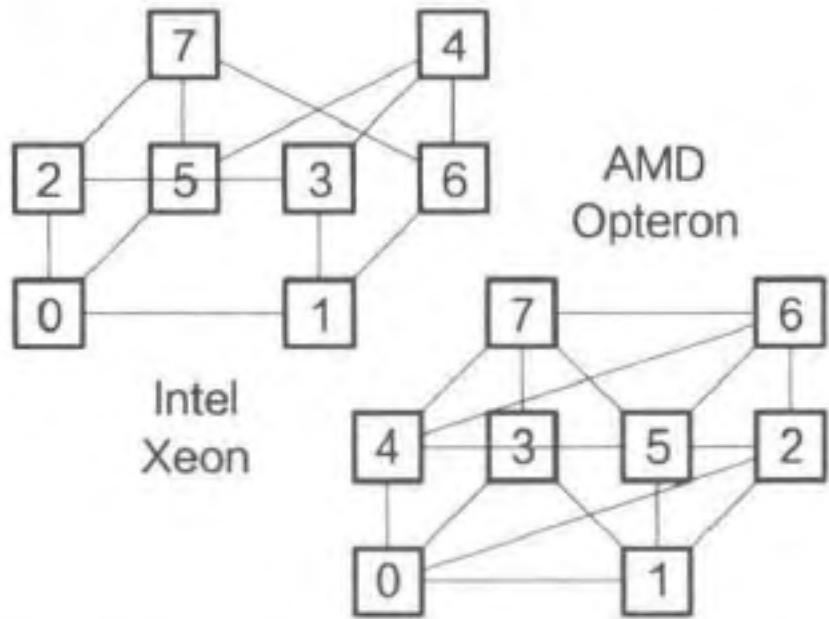


图 5.16 Intel 与 AMD 示例服务器拓扑结构图

NUMA 架构对于操作系统不是透明的，因此为了应对下层硬件的非一致内存访问，操作系统应当分配并管理好硬件资源，尽可能避免应用中频繁进行远程内存访问造成的性能损耗。

5.3.6 NUMA 架构与内存分配

面向 NUMA 架构，操作系统在分配内存资源时需要采取针对性措施，尽可能使得应用进程中的访存操作均是访问本地内存，从而避免因频繁的远程内存访问而造成严重的性能问题。

现代操作系统一般采用两种策略来进行优化。一方面，操作系统可以为上层应用提供 NUMA 感知的内存分配接口，让上层应用显式地指定分配内存所在位置。具体而言，有的应用进程会绑定特定的 CPU 核心执行。对于这些应用进程，其可以使用操作系统提供的接口，显式地指定这些线程分配的内存均为其绑定的 CPU 核心所处节点的本地内存。另一方面，对于没有显式使用这些接口的应用，操作系统需要根据当前线程运行的节点，尽可能地将内存分配在本地节点，从而避免远程内存访问。当节点本地内存剩余空间不够时，操作系统应当优先考虑临近节点是否还有空余的内存空间可以分配。

以 Linux 为例，它针对 NUMA 环境提供了三种内存分配模式：绑定模式、优先模式、交错模式。顾名思义，绑定模式指将从指定的 NUMA 节点上分配内存。应用可以使用该模式选择在自己运行的 NUMA 节点上分配本地内存。优先模式则在分配失败时，尝试从指定节点最近的节点上分配内存。交错模式则会从应用给定的节点中以页为粒度交错地分配内存。如果应用没有选用任何模式，操作系统则会采用优先模式帮助应用在其运行的节点及邻近的节点上分配内存。

Linux 还提供了 libnuma 库，其将 NUMA 相关的系统调用包装成直观且易用的接口，应用进程可以方便地用其直接在特定的节点上分配内存。代码片段 5.6 通过一个简单的例子介绍应用进程如何使用该库。libnuma 提供 numa_alloc_onnode 接口，用于在指定的 NUMA 节点上分配固定大小的内存。其接收两个参数，第一个参数为分配的内存大

小，第二个参数为指定的 NUMA 节点编号。当应用需要在本地节点分配内存时，它可以先获取自己所在的 NUMA 节点编号，然后使用该接口分配内存。如代码所示，应用首先通过 `sched_getcpu` 接口获取当前的 CPU 核心编号，然后通过 `numa_node_of_cpu` 接口获取 CPU 核心编号对应的 NUMA 节点编号。然后，它将刚刚获取的 NUMA 节点编号作为 `numa_alloc_onnode` 的参数，在本地节点上分配内存。此外，`libnuma` 还提供了更为方便的 `numa_alloc_local` 用于在本地节点分配内存，省去了获取本地 NUMA 节点编号的流程。`libnuma` 也提供 `numa_alloc_interleaved` 接口，用于使用交错模式分配内存。需要注意的是，通过以上接口分配的内存均需使用 `numa_free` 接口进行释放。

代码片段 5.6 libnuma 使用示例

```

1 #define _GNU_SOURCE
2 #include <numa.h>
3 #include <sched.h>
4 #include <stdio.h>
5
6 int main(int argc, char *argv[])
7 {
8     int cpu = sched_getcpu();
9     int node = numa_node_of_cpu(cpu);
10    int *mem_0 = numa_alloc_onnode(sizeof(int), node);
11    int *mem_1 = numa_alloc_local(sizeof(int));
12    int *mem_2 = numa_alloc_interleaved(sizeof(int));
13
14    *mem_0 = 0; *mem_1 = 0; *mem_2 = 0;
15    numa_free(mem_0, sizeof(int));
16    numa_free(mem_1, sizeof(int));
17    numa_free(mem_2, sizeof(int));
18    return 0;
19 }

```

5.4 思考题

1. 伙伴系统中的释放操作在最理想和最坏情况下的时间复杂度各是多少？
2. 请尝试构造一种页访问顺序，使得 FIFO 策略触发 Belady 异常。
3. 请参照 FIFO 等策略的示例，给出时钟算法在相同页访问顺序下发生缺页异常的次数。
4. 当下，服务器的内存通常已经达到上百 GB 甚至更大。请思考，今天换页还重要吗？若数据中心的网络数据传输速度超过磁盘读写速度，那么请提出一种可能的更加高效的换页设计。
5. 通过前面的学习，我们知道操作系统可以通过伙伴系统配合 SLAB 分配器进行物理内存分配，优秀的物理内存分配器还需要考虑到性能隔离，比如如何尽量避免缓存冲突等。结合 Intel CAT 和 ARM MPAM 硬件特性，提出一种可能的操作系统利用这些特